

Week 2 - Day 10 Supplementary: Time Complexity

The following function helps you test your code:

```
# RUN THIS BLOCK OF CODE BEFORE RUNNING ANY CELLS
def check_test(actual, expected):
    if expected != actual:
        print(f"Function should return the value {expected}, it is returning the value {actual}")
    else:
        print(f"Congratulations, the test case passed!")
```

The following function helps you **time** your code:

```
import time

def timer(func, *args):
    start = time.perf_counter()
    result = func(*args)
    args_str = ', '.join([str(arg) for arg in args])
    end = time.perf_counter()
    print(f"Time to run function {func.__name__}({args_str}): {end - start:.5f} seconds")
    return result

result = timer(sorted, range(1000000))
```

Important: Ask a TA to check your work!

Exercise 1: Nested Loop [] []

Consider the following loop

```
for i in range(len(n)):
    for j in range(len(n)):
        for k in range(len(n)):
            print(i, k, j)
```

If you run the above code with $n = 1000$ how many times does the function call print ?

Exercise 2: Fibonacci [] []

Let's go back to fibonacci for a second. Assume `fib(n)` only gets `n >= 0`.

```
def fib(n):  
    if n <= 1:  
        return 1  
    return fib(n-1) + fib(n-2)
```

Try running values `fib(10)`, `fib(15)`, `fib(20)`, ... and see how long it takes for each of the cases.

```
fib10 = timer(fib, 10) # The `timer` function calls the `fib`  
                        # function with the argument 10 for you  
fib15 = timer(fib, 15)  
fib20 = timer(fib, 20)  
fib25 = timer(fib, 25)  
fib30 = timer(fib, 30)  
fib35 = timer(fib, 35)  
  
# TODO: Do you think we can finish `fib(100)`? Why or why not?  
timer(fib, 100)
```

Explain your observation below in terms of time complexity. How many recursive function calls do we need to solve for each of the cases above?

Exercise 3: Loop [] []

Write the time complexity of the following program in terms of n .

```
def loop(k):  
    for i in range(k):  
        print(i)  
  
for j in range(n):  
    loop(j)
```

Exercise 4: Multiple Choice [] []

4.1. What is the time complexity of an algorithm that checks if an item exists in an unsorted list?

- A. $O(n)$
- B. $O(n^2)$
- C. $O(1)$
- D. $O(\log(n))$

4.2. If an algorithm's time complexity is $O(n^2)$ and it takes 1 second to process 1000 elements, approximately how long will it take to process 2000 elements?

- A. 2 seconds
- B. 4 seconds
- C. 8 seconds
- D. 16 seconds

4.3. What is the time complexity of finding the smallest or largest item in an unsorted list?

- A. $O(n)$
- B. $O(n^2)$
- C. $O(1)$
- D. $O(\log(n))$

4.4. What is the time complexity of a recursive Fibonacci algorithm?

Exercise 5: Iterative vs Recursive [] []

5.1. Write a Python function to calculate the factorial of a number ($n!$) using both iterative and recursive approaches.

```
# TODO: Write the iterative and recursive factorial functions

def factorial_iterative(n):
    # Your code here
    pass

def factorial_recursive(n):
    # Your code here
    pass

fac1000_iter = timer(factorial_iterative, 1000)
fac1000_rec = timer(factorial_recursive, 1000)
```

5.2. What is the time complexity of each approach?

Exercise 6: Time Complexity of Functions [] []

6.1.a. Write a Python function that takes in a list of numbers and returns a new list with only the unique numbers (i.e., remove duplicates).

```
# TODO: Finish remove_duplicates function

def remove_duplicates(lst):
    # Your code here
    return []

length = 1000000
dup_list = list(range(length)) + list(range(length))
uniq_list = remove_duplicates(dup_list)
print('Test passed:', sorted(uniq_list) == sorted(range(length)))
```

6.1.b. What is the time complexity of your function?

6.2.a. Write a Python function that accepts two strings and checks if one is a permutation of the other.

```
# TODO: Finish is_permutation function
def is_permutation(str1, str2):
    # Your code here
    pass

print(is_permutation("abc", "cba"))
print(is_permutation("abc", "ab"))
```

6.2.b. What is the time complexity of your function?

Exercise 7: Find Unique Element [] [] [] []

You are given a **large, sorted** list of integers, where each integer **appears exactly twice** except for one element that **appears only once**.

Implement a function called `find_unique_element` that takes in this sorted list as input and returns the unique element.

For example, given the sorted list `[1, 1, 2, 2, 3, 3, 4, 5, 5, 6, 6]`, the function should return `4` since it is the only element that appears once in the list.

Let's start with a linear time solution below:

```
#TODO: Finish find_unique_element function

def find_unique_element(arr):
    # Your code here
    pass

check_test(find_unique_element([1, 1, 2, 2, 3, 3, 4, 5, 5, 6, 6]), 4)
check_test(find_unique_element([1]), 1)
check_test(find_unique_element([3, 4, 4]), 3)
check_test(find_unique_element([1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6]), 6)
```

Challenge: Can you implement this function in $O(\log(n))$ time complexity?

Hint: Use binary search. What happens when the middle element is the same as the left or the right element? What happens when the middle element is different from both the left and right elements?

```
# TODO: Implement find_unique_element_binary_search function

def find_unique_element_binary_search(arr):
    # Your code here
    pass

check_test(find_unique_element_binary_search([1, 1, 2, 2, 3, 3, 4, 5,
5, 6, 6]), 4)
check_test(find_unique_element_binary_search([1]), 1)
check_test(find_unique_element_binary_search([3, 4, 4]), 3)
check_test(find_unique_element_binary_search([1, 1, 2, 2, 3, 3, 4, 4,
5, 5, 6]), 6)
```

Congratulations 🎉